

INF 491 — Tópicos Especiais: Introdução à Computação Musical — Trabalho Prático

Informações Básicas

Tema: Síntese musical e renderização offline de áudio

Linguagem: Python

Forma de realização: individual ou em duplas

Data de publicação: 22/05/2026

Prazo de entrega: 22 dias

Data limite de entrega: 13/06/2026, às 23:59:59

Modo de entrega: Tarefa do Moodle

1. Visão geral

Neste trabalho, você deverá desenvolver uma versão simplificada de uma estação de produção musical digital, chamada **MicroDAW**.

Uma DAW (*Digital Audio Workstation*) é um ambiente usado para criação, edição, processamento e reprodução de áudio. Neste trabalho, a proposta é implementar uma versão reduzida desse conceito, voltada especificamente para **síntese musical programada em Python**.

O sistema desenvolvido deverá permitir que o usuário crie ou personalize um **instrumento sintético**, escolhendo parâmetros que afetam seu timbre. Em seguida, o usuário deverá selecionar um arquivo .mid, realizar a sintetização offline da música e, por fim, ouvir o resultado gerado com o som do instrumento configurado.

A sintetização será considerada **offline** porque o áudio não precisa ser produzido em tempo real. O programa pode primeiro processar todos os eventos musicais, gerar a waveform completa e somente depois reproduzir ou salvar o resultado.

2. Ideia central do trabalho

A ideia principal é construir uma aplicação capaz de:

1. permitir a customização de timbres sintéticos;
2. criar instrumentos a partir de parâmetros de síntese;
3. carregar arquivos MIDI fornecidos;
4. processar as notas presentes no arquivo MIDI;
5. gerar uma waveform final correspondente à música sintetizada;
6. reproduzir e/ou salvar o áudio gerado;
7. opcionalmente, gerar visualizações do sinal usando ferramentas fornecidas.

O foco do trabalho não é reproduzir perfeitamente instrumentos reais. O objetivo é explorar, de forma criativa e tecnicamente fundamentada, os conceitos estudados no capítulo de síntese musical, incluindo osciladores, formas de onda, envelopes, modulação, filtros, timbre e composição de sinais.

3. Arquivos fornecidos

Para sua conferência, dentro do arquivo .zip que você baixou deve haver o seguinte conteúdo:

- Na pasta raiz, este enunciado:
 - enunciado_microdaw.pdf
- Pasta python, contendo os arquivos (explicados mais adiante):
 - microplayer.py
 - microsynth.py
 - microview.py
- Pasta midi, contendo os arquivos:
 - AquarelaBrasil.mid
 - ave-maria.mid
 - FMP_C1_F12_Bach_BWV846.mid
 - HotelCalifornia.mid

3.1 Explicação sobre os arquivos MIDI

Serão fornecidos alguns arquivos .mid para uso no trabalho. Alguns são mais complexos e/ou pesados que outros. O mais simples e um bom teste inicial para testar todo o desenvolvimento é o arquivo `ave-maria.mid`.

A implementação final deverá funcionar com os arquivos MIDI fornecidos pelo professor. A equipe também poderá testar o programa com outros arquivos MIDI de sua escolha, desde que a versão entregue funcione corretamente com os arquivos oficiais disponibilizados.

O programa deve ser capaz de lidar com arquivos contendo **múltiplos canais, partes ou vozes**. Cada voz deverá ser processada e posicionada corretamente no tempo, respeitando os offsets e durações das notas.

4. Interface da solução

A solução de interface fica livre, a cargo da equipe.

A equipe poderá desenvolver, por exemplo:

- uma interface em linha de comando;
- uma interface baseada em menus textuais;
- uma interface em notebook Jupyter;
- uma interface gráfica simples;
- uma interface Web local;
- outra solução equivalente, desde que permita configurar os parâmetros necessários e executar o processamento.

A interface não será avaliada por sofisticação visual, mas deve permitir que o usuário utilize o programa de forma clara e organizada.

5. Instrumentos sintéticos

A equipe deverá utilizar obrigatoriamente a classe `Instrumento`, que será fornecida parcialmente como parte do material do trabalho.

Essa classe deverá representar um instrumento sintético configurável, isto é, um objeto capaz de gerar notas musicais a partir de parâmetros como frequência, duração, amplitude e características de timbre.

Serão fornecidos alguns modelos iniciais de instrumentos, que poderão ser usados como referência. No entanto, cada equipe deverá criar pelo menos **dois instrumentos próprios**, definidos a partir de escolhas próprias de parâmetros.

É permitida pesquisa na Web para estudar características gerais de instrumentos, síntese sonora, envelopes, formas de onda, filtros e estratégias de criação de timbre. Entretanto, as decisões adotadas devem ser compreendidas e explicadas pela equipe.

6. Processamento de múltiplas vozes/canais

O sistema deve ser capaz de processar músicas com múltiplas vozes.

Cada voz deve gerar seu próprio sinal intermediário. As notas de cada voz devem ser sintetizadas e posicionadas corretamente no tempo, considerando:

- frequência da nota;
- instante de início;
- duração;
- intensidade, quando disponível;
- instrumento associado à voz.

Ao final, o programa deve gerar uma única waveform final, correspondente à soma das vozes processadas, pronta para reprodução ou exportação.

A equipe deverá tomar cuidado para evitar distorções indesejadas por excesso de amplitude. Estratégias de normalização, controle de ganho ou mixagem poderão ser utilizadas.

7. Uso de NumPy

A implementação deve utilizar operações vetorizadas com NumPy sempre que possível.

O uso de NumPy é especialmente esperado na geração de sinais, criação de vetores de tempo, aplicação de envelopes, modulações, filtros, combinação de buffers e normalização do áudio.

Laços de repetição podem ser usados quando fizerem sentido, especialmente para percorrer eventos musicais, vozes ou instrumentos. No entanto, a geração e manipulação dos sinais de áudio não deve ser feita amostra por amostra de maneira desnecessária.

8. Biblioteca de visualização

Será fornecido um arquivo `microview.py` contendo funções auxiliares de visualização e análise de sinais.

Essa biblioteca poderá ser usada pela equipe para gerar gráficos como:

- forma de onda;
- espectro de frequências;
- espectrograma;
- envelope do sinal;
- energia por voz;
- visualizações tipo piano roll.

A implementação das funções de visualização não faz parte do núcleo obrigatório, **mas desejável**, do trabalho, pois o foco principal está na síntese, no processamento dos eventos musicais e na geração do áudio final.

9. Escopo da implementação

O trabalho será desenvolvido a partir de três arquivos-base fornecidos pelo professor:

- `microsynth.py`: contém funções e classes relacionadas à geração e transformação de sinais sonoros;
- `microplayer.py`: contém estruturas para leitura de arquivos MIDI, organização dos eventos musicais, processamento das vozes e geração da waveform final;
- `microview.py`: contém funções auxiliares de visualização e análise dos sinais gerados.

Algumas partes desses arquivos serão fornecidas prontas, algumas deverão ser evoluídas a partir de uma versão básica fornecida e outras deverão ser implementadas integralmente pela equipe.

O arquivo `microview.py` será fornecido integralmente pronto e poderá ser usado livremente pela equipe. Já os arquivos `microsynth.py` e `microplayer.py` conterão trechos completos, trechos parciais e trechos a serem implementados.

A definição detalhada dessas partes será especificada nas seções seguintes do enunciado.

Também são pedidas algumas evoluções para a equipe fazer, após a implementação da parte obrigatória. Estas evoluções estão descritas na seção “Evoluções naturais”.

9.1. Partes fornecidas prontas

Algumas funções serão fornecidas integralmente prontas nos arquivos-base do trabalho. Essas funções poderão ser usadas diretamente pela equipe, sem necessidade de reimplementação.

Mesmo quando uma função for fornecida pronta, é importante compreender seu papel dentro do sistema, pois ela poderá ser chamada por outras partes que deverão ser implementadas ou completadas pela equipe.

ARQUIVO `MICROSYNTH.PY`

O arquivo `microsynth.py` contém funções relacionadas à geração, transformação e processamento de sinais sonoros.

As seguintes funções serão fornecidas prontas:

`gera_tempo`

```
def gera_tempo(dur: float, sr: int) -> np.ndarray:
```

Gera um vetor de tempo discreto para representar a duração de um sinal de áudio.

Essa função cria um array NumPy contendo os instantes de tempo correspondentes às amostras de um sinal digital. Ela é útil para gerar osciladores, envelopes, modulações e qualquer outro sinal dependente do tempo.

Parâmetros:

- `dur`: duração do sinal, em segundos.
- `sr`: taxa de amostragem, em amostras por segundo.

Retorno:

- Um array NumPy contendo os instantes de tempo do sinal.

Exemplo conceitual:

Para uma duração de 2 segundos e taxa de amostragem de 44100 Hz, o vetor gerado terá aproximadamente 88200 amostras.

`midi2freq`

```
def midi2freq(n: int) -> float:
```

Converte o número de uma nota MIDI para sua frequência correspondente em Hertz.

A função considera o padrão usual em que a nota MIDI 69 corresponde à frequência de 440 Hz, isto é, a nota Lá 4 (A4).

Parâmetros:

- `n`: número MIDI da nota, inteiro no intervalo de 0 a 127.

Retorno:

- A frequência correspondente à nota MIDI, em Hertz.

Observação:

Essa função será importante para transformar eventos musicais extraídos de arquivos MIDI em frequências que possam ser usadas pelos osciladores e instrumentos sintéticos.

`fm_time_warp`

```
def fm_time_warp(
    y: np.ndarray,
    t: np.ndarray,
    f_m: float,
    I: float
) -> np.ndarray:
```

Aplica uma forma aproximada de modulação de frequência a um sinal já existente, usando uma técnica de deformação temporal (*time warping*).

A ideia é alterar ligeiramente os instantes de amostragem do sinal original, produzindo um efeito semelhante a vibrato ou modulação de frequência, dependendo dos parâmetros utilizados.

Parâmetros:

- `y`: sinal original, representado como array NumPy.
- `t`: vetor de tempo associado ao sinal `y`.
- `f_m`: frequência da modulação, em Hertz.
- `I`: intensidade da deformação temporal, em segundos.

Retorno:

- Um novo array NumPy contendo o sinal após a aplicação da deformação temporal.

Observação:

Essa função não gera um sinal FM do zero. Ela recebe um sinal já existente e altera sua evolução temporal, funcionando como uma aproximação útil para efeitos como vibrato.

filtro

```
def filtro(
    y: np.ndarray,
    cutoff_low: float = None,
    cutoff_high: float = None,
    metodo: str = 'filtfilt',
    sr: int = 44100
) -> np.ndarray:
```

Aplica um filtro digital ao sinal de entrada.

Dependendo dos parâmetros informados, a função pode atuar como filtro passa-baixa, passa-alta ou passa-banda:

- se apenas `cutoff_high` for informado, aplica um filtro passa-baixa;
- se apenas `cutoff_low` for informado, aplica um filtro passa-alta;
- se `cutoff_low` e `cutoff_high` forem informados, aplica um filtro passa-banda.

A função utiliza um filtro Butterworth de ordem 4 e normaliza o sinal resultante.

Parâmetros:

- `y`: sinal de entrada, representado como array NumPy.
- `cutoff_low`: frequência de corte inferior, em Hertz. Pode ser `None`, dependendo do tipo de filtro desejado.
- `cutoff_high`: frequência de corte superior, em Hertz. Pode ser `None`, dependendo do tipo de filtro desejado.
- `metodo`: método de aplicação do filtro. Pode ser `'lfilter'` ou `'filtfilt'`.
- `sr`: taxa de amostragem, em amostras por segundo.

Retorno:

- Um array NumPy contendo o sinal filtrado e normalizado.

Observações:

- As frequências de corte devem estar no intervalo válido, respeitando o limite de Nyquist ($sr/2$).
 - O filtro poderá ser usado para modificar o timbre dos instrumentos sintéticos.
-

ARQUIVO MICROPLAYER.PY

O arquivo `microplayer.py` contém estruturas relacionadas à leitura de arquivos MIDI, organização de eventos musicais, processamento de vozes e geração da waveform final.

A seguinte função será fornecida pronta:

freq2midi

```
def freq2midi(f):
```

Converte uma frequência em Hertz para uma representação de nota MIDI usando a biblioteca `music21`.

Essa função é diferente da função `freq2midi` que deverá ser implementada no arquivo `microsynth.py`. Aqui, a conversão é feita com apoio da biblioteca `music21`, retornando tanto o número MIDI quanto o nome da nota.

Parâmetros:

- `f`: frequência, em Hertz.

Retorno:

- Uma tupla contendo:
 - o número MIDI correspondente à frequência;

- o nome da nota com oitava, como C4, F#3, A4 etc.

Observação:

Essa função poderá ser útil para inspeção, depuração e visualização das frequências processadas pelo sistema.

ARQUIVO MICROVIEW.PY

O arquivo `microview.py` será fornecido integralmente pronto. Ele contém funções auxiliares de visualização e análise dos sinais gerados pelo MicroDAW.

A equipe poderá utilizar essas funções livremente, sem necessidade de reimplementá-las. O uso dessas visualizações é recomendado para compreender e apresentar melhor o comportamento do sistema.

`plot_sinais`

```
def plot_sinais(
    ys: list[np.ndarray],
    labels: list[str] = None,
    sr: int = 44100,
    fatia: int = None,
    titulo: str = "Comparação de Sinais",
    figsize: tuple[int, int] = (12, 6)
) -> None:
```

Plota um ou mais sinais de áudio em gráficos empilhados, compartilhando o mesmo eixo de tempo.

Essa função é útil para comparar sinais diferentes, visualizar formas de onda ou observar trechos específicos de uma waveform.

Parâmetros:

- `ys`: lista de arrays NumPy, cada um representando um sinal 1D.
- `labels`: lista de rótulos para os sinais. Se não for informada, rótulos genéricos serão usados.
- `sr`: taxa de amostragem, usada para converter amostras em tempo.
- `fatia`: quantidade máxima de amostras a exibir. Se for `None`, exibe o sinal inteiro.
- `titulo`: título da figura.
- `figsize`: tamanho da figura.

Retorno:

- `None`. A função apenas exibe o gráfico.
-

`plot_envelope`

```
def plot_envelope(t: np.ndarray, env: np.ndarray) -> None:
```

Plota um envelope ao longo do tempo.

Essa função pode ser usada, por exemplo, para visualizar um envelope ADSR gerado pela implementação da equipe.

Parâmetros:

- `t`: vetor de tempo, como array NumPy.
- `env`: vetor contendo os valores do envelope. Deve ter o mesmo tamanho de `t`.

Retorno:

- `None`. A função apenas exibe o gráfico.
-

`plot_espectrograma`

```
def plot_espectrograma(y, sr=44100):
```

Gera e exibe o espectrograma de um sinal.

O espectrograma mostra como o conteúdo de frequências do sinal varia ao longo do tempo, sendo útil para observar mudanças de timbre, presença de harmônicos, efeitos de filtros e modulações.

Parâmetros:

- y: sinal de entrada.
- sr: taxa de amostragem.

Retorno:

- None. A função apenas exibe o gráfico.
-

plot_fft

```
def plot_fft(y, sr=44100, titulo="Espectro"):
```

Calcula e exibe o espectro de frequências de um sinal usando a Transformada Rápida de Fourier.

Essa função é útil para analisar quais frequências estão presentes em um som e como diferentes formas de onda, filtros e modulações afetam o espectro.

Parâmetros:

- y: sinal de entrada.
- sr: taxa de amostragem.
- titulo: título do gráfico.

Retorno:

- None. A função apenas exibe o gráfico.
-

plot_envelope_sinal

```
def plot_envelope_sinal(y, sr=44100, janela=1024):
```

Estima e exibe uma aproximação do envelope de amplitude de um sinal.

A função calcula o valor absoluto do sinal e aplica uma suavização por média móvel, permitindo observar a evolução global da amplitude ao longo do tempo.

Parâmetros:

- y: sinal de entrada.
- sr: taxa de amostragem.
- janela: tamanho da janela usada na suavização por média móvel.

Retorno:

- None. A função apenas exibe o gráfico.
-

plot_energia

```
def plot_energia(buffers):
```

Calcula e exibe a energia de cada voz a partir de um conjunto de buffers.

Essa função é útil para observar o equilíbrio entre vozes antes ou depois da mixagem.

Parâmetros:

- buffers: array contendo os sinais das vozes. Espera-se uma estrutura em que cada linha corresponde a uma voz.

Retorno:

- None. A função apenas exibe o gráfico.

freq2midi_int

```
def freq2midi_int(f: float) -> int:
```

Converte uma frequência em Hertz para o número MIDI inteiro mais próximo.

Essa função é usada internamente por algumas funções de visualização, especialmente para gerar visualizações do tipo piano roll.

Parâmetros:

- f: frequência, em Hertz.

Retorno:

- Número MIDI inteiro correspondente à frequência aproximada.
-

midi2note_name

```
def midi2note_name(midi: int) -> str:
```

Converte um número MIDI para o nome da nota correspondente, incluindo a oitava.

A função usa a biblioteca music21 para obter nomes como C4, D#5, A3 etc.

Parâmetros:

- midi: número MIDI da nota.

Retorno:

- String com o nome da nota e sua oitava.
-

plot_pianoroll

```
def plot_pianoroll(eventos, voz=0):
```

Gera uma visualização tipo piano roll para uma única voz.

Cada evento musical é desenhado como uma linha horizontal, indicando sua nota, instante de início, duração e intensidade.

Parâmetros:

- eventos: lista de eventos musicais. Cada evento deve seguir o formato (f, inicio, dur, amp):
 - f: frequência da nota;
 - inicio: instante de início, em segundos;
 - dur: duração da nota, em segundos;
 - amp: intensidade relativa da nota.
- voz: identificador numérico da voz, usado no título do gráfico.

Retorno:

- None. A função apenas exibe o gráfico.
-

plot_pianoroll_multivoz

```
def plot_pianoroll_multivoz(lista_eventos, nomes_vozes=None):
```

Gera uma visualização tipo piano roll para múltiplas vozes.

Essa função permite visualizar simultaneamente os eventos musicais de várias partes do arquivo MIDI, ajudando a compreender a estrutura temporal da música.

Parâmetros:

- `lista_eventos`: lista contendo listas de eventos musicais. Cada sublista representa uma voz.
- `nomes_vozes`: lista opcional com os nomes das vozes, usada para gerar a legenda.

Retorno:

- None. A função apenas exibe o gráfico.
-

9.2. Partes a evoluir a partir do código fornecido

Algumas funções serão fornecidas parcialmente prontas nos arquivos-base do trabalho. As partes faltantes dessas funções devem ser implementadas pela equipe, respeitando a assinatura das funções/métodos e a saída proposta.

É importante compreender seu papel dentro do sistema, pois ela poderá ser chamada por outras partes que deverão ser implementadas ou completadas pela equipe.

ARQUIVO `MICROSYNTH.PY`

O arquivo `microsynth.py` concentra as funções ligadas diretamente à síntese e à transformação de sinais sonoros.

As seguintes funções e classe deverão ter suas implementações completadas pela equipe:

ADSR

```
def adsr(
    dur: float,
    sr: int,
    A: float = 0.1,
    D: float = 0.1,
    S: float = 0.6,
    R: float = 0.2
) -> np.ndarray:
```

A função `adsr` deve gerar um envelope do tipo ADSR (*Attack, Decay, Sustain, Release*), usado para controlar a evolução da amplitude de uma nota ao longo do tempo.

O envelope produzido por essa função será aplicado posteriormente ao sinal gerado por um oscilador ou instrumento sintético. Portanto, ele deve ter o mesmo número de amostras da nota que será sintetizada.

O modelo ADSR utilizado neste trabalho considera que a duração total da nota corresponde à duração total do envelope. Assim, as fases de ataque, decaimento, sustentação e liberação devem caber dentro do intervalo definido por `dur`.

Parâmetros:

- `dur`: duração total do envelope, em segundos.
- `sr`: taxa de amostragem, em amostras por segundo.
- `A`: duração da fase de ataque, em segundos.
- `D`: duração da fase de decaimento, em segundos.
- `S`: nível da fase de sustentação, entre 0 e 1.
- `R`: duração da fase de liberação, em segundos.

Retorno:

- Um array NumPy contendo o envelope ADSR.

Comportamento esperado:

A função deve:

- calcular o número total de amostras do envelope a partir de `dur` e `sr`;
- converter os tempos `A`, `D` e `R` de segundos para amostras;
- calcular automaticamente a duração da fase de sustentação;
- construir as quatro fases do envelope usando operações vetorizadas com NumPy;
- concatenar as fases para formar o envelope final;
- garantir que o array retornado tenha exatamente `round(sr * dur)` amostras;
- lidar adequadamente com notas curtas, isto é, casos em que $A + D + R > dur$.

Observações:

A implementação deve evitar processamento amostra por amostra. Use funções vetorizadas do NumPy, como criação de vetores, concatenação e operações sobre arrays.

Caso a soma das durações de ataque, decaimento e liberação seja maior do que a duração total da nota, a equipe deverá adotar uma estratégia para ajustar as fases do envelope, preservando um comportamento coerente e garantindo o tamanho final esperado.

Exemplo de uso esperado:

```
env = adsr(  
    dur=2.0,  
    sr=44100,  
    A=0.1,  
    D=0.2,  
    S=0.6,  
    R=0.4  
)
```

Nesse exemplo, env deve ser um array NumPy com 88200 amostras, correspondente a 2 segundos de áudio com taxa de amostragem de 44100 Hz.

SINETIZA

```
def sintetiza(  
    f: float,  
    forma: str = 'senoide',  
    n: int = 5,  
    dur: float = 2.0,  
    sr: int = 44100,  
    fase: float = 0.0,  
    unidade_fase: str = 'graus',  
    retorna_t: bool = False  
) -> np.ndarray | tuple[np.ndarray, np.ndarray]:
```

A função sintetiza deve gerar um sinal sonoro periódico a partir de uma frequência fundamental e de uma forma de onda escolhida.

Essa função será usada como uma das bases para a criação dos instrumentos sintéticos do MicroDAW. Ela deve permitir a geração de sinais simples, como senoides, e também de sinais mais ricos em harmônicos, como ondas quadradas, triangulares e dente de serra.

Parâmetros:

- f: frequência fundamental do sinal, em Hertz.
- forma: tipo de forma de onda a ser gerada. Os valores aceitos são:
 - 'senoide';
 - 'quadrada';
 - 'triangular';
 - 'dente'.
- n: número máximo de harmônicos a serem considerados na construção da forma de onda.
- dur: duração do sinal, em segundos.
- sr: taxa de amostragem, em amostras por segundo.
- fase: fase inicial do sinal.
- unidade_fase: unidade usada para interpretar o parâmetro fase. Os valores aceitos são:
 - 'graus';
 - 'rad'.
- retorna_t: indica se a função deve retornar apenas o sinal gerado ou também o vetor de tempo.

Retorno:

- Se retorna_t = False, a função deve retornar apenas o sinal gerado, como um array NumPy.
- Se retorna_t = True, a função deve retornar uma tupla (t, y), em que:
 - t é o vetor de tempo;
 - y é o sinal gerado.

Comportamento esperado:

A função deve:

- validar os parâmetros recebidos;
- gerar um vetor de tempo compatível com a duração e a taxa de amostragem informadas;
- converter a fase inicial para radianos, quando necessário;
- gerar a forma de onda solicitada;
- normalizar o sinal final para evitar amplitudes excessivas;
- retornar apenas o sinal ou o par (t, y) , de acordo com o valor de `retorna_t`.

A implementação das formas de onda deverá seguir as seguintes ideias:

- a forma 'senoide' deve ser gerada diretamente como uma única componente senoidal na frequência f ;
- a forma 'quadrada' deve ser construída por soma de harmônicos ímpares;
- a forma 'triangular' deve ser construída por soma de harmônicos ímpares, com queda de amplitude mais rápida e alternância de sinal;
- a forma 'dente' deve ser construída por soma de harmônicos inteiros.

Para as formas compostas por harmônicos, a função **não deve incluir componentes cuja frequência seja maior ou igual à frequência de Nyquist**, isto é, $sr / 2$.

Observações:

A implementação deve utilizar operações vetorizadas com NumPy sempre que possível.

O uso de laços é aceitável para percorrer os harmônicos, mas a geração dos sinais deve ser feita por operações sobre arrays, e não por processamento amostra por amostra.

A normalização final deve preservar a forma geral do sinal e apenas ajustar sua amplitude para uma faixa adequada.

Exemplo de uso esperado:

```
t, y = sintetiza(
    f=440,
    forma='quadrada',
    n=10,
    dur=2.0,
    sr=44100,
    fase=0.0,
    unidade_fase='graus',
    retorna_t=True
)
```

Nesse exemplo, t deve ser um vetor de tempo com aproximadamente 88200 amostras, e y deve conter o sinal correspondente a uma onda quadrada sintética de 440 Hz, construída a partir de harmônicos ímpares.

CLASSE INSTRUMENTO

```
class Instrumento:
```

A classe `Instrumento` representa um instrumento sintético configurável dentro do MicroDAW.

Cada objeto dessa classe deve encapsular os parâmetros necessários para gerar notas musicais com um determinado timbre. A ideia é que o instrumento combine uma fonte sonora básica, como uma forma de onda ou síntese FM, com transformações e controles como envelope ADSR, modulação de amplitude, modulação de frequência, filtragem e intensidade.

A classe será fornecida parcialmente implementada. O construtor será fornecido pronto, mas o método `gerar_nota` deverá ser completado pela equipe.

Construtor

```
def __init__(
    self,
    nome: str,
    forma: str,
    n_harm: int,
    adsr_params: tuple,
```

```
fase: float = 0.0,
unidade_fase: str = 'graus',
am_params: dict = None,
fm_params: dict = None,
filtro_params: dict = None,
efeito_params: dict = None
):
```

O construtor recebe os parâmetros que definem o comportamento sonoro do instrumento.

Parâmetros:

- nome: nome do instrumento sintético.
- forma: forma de geração sonora usada como base. Os valores previstos são:
 - 'senoide';
 - 'quadrada';
 - 'triangular';
 - 'dente';
 - 'fm'.
- n_harm: número máximo de harmônicos usados quando a forma de onda for construída por soma de harmônicos.
- adsr_params: tupla com os parâmetros do envelope ADSR, no formato (A, D, S, R).
- fase: fase inicial do sinal.
- unidade_fase: unidade usada para interpretar a fase. Os valores previstos são:
 - 'graus';
 - 'rad'.
- am_params: dicionário com parâmetros para modulação de amplitude, ou None caso AM não seja usada.
- fm_params: dicionário com parâmetros para síntese ou modulação de frequência, ou None caso FM não seja usada.
- filtro_params: dicionário com parâmetros para filtragem, ou None caso nenhum filtro seja usado.
- efeito_params: dicionário com parâmetros para efeitos opcionais, ou None caso nenhum efeito seja usado.

Método gerar_nota

```
def gerar_nota(
    self,
    f: float,
    dur: float,
    sr: int = 44100,
    amp: float = 1.0,
    retorna_t: bool = False
) -> np.ndarray | tuple[np.ndarray, np.ndarray]:
```

O método gerar_nota deve gerar uma nota musical sintetizada usando os parâmetros armazenados no instrumento.

Esse método é o núcleo da classe Instrumento. Ele deve combinar as funções implementadas ou fornecidas no arquivo microsynth.py para produzir o sinal correspondente a uma única nota.

Parâmetros:

- f: frequência da nota, em Hertz.
- dur: duração da nota, em segundos.
- sr: taxa de amostragem, em amostras por segundo.
- amp: fator de intensidade da nota. Esse valor normalmente virá da intensidade da nota extraída do arquivo MIDI.
- retorna_t: indica se a função deve retornar apenas o sinal gerado ou também o vetor de tempo.

Retorno:

- Se retorna_t = False, o método deve retornar apenas o sinal da nota, como um array NumPy.
- Se retorna_t = True, o método deve retornar uma tupla (t, y), em que:
 - t é o vetor de tempo;
 - y é o sinal da nota sintetizada.

Comportamento esperado

O método gerar_nota deve implementar o seguinte fluxo de processamento:

1. Validar a forma de geração sonora definida em `self.forma`.
2. Gerar o sinal base da nota:
 - se `self.forma` for 'senoide', 'quadrada', 'triangular' ou 'dente', a função sintetiza deve ser usada;
 - se `self.forma` for 'fm', a função `fm` deve ser usada.
3. Aplicar envelope ADSR, caso `self.adsr_params` não seja `None`.
4. Aplicar modulação de amplitude, caso `self.am_params` não seja `None`.
5. Aplicar modulação de frequência por *time warping*, caso:
 - `self.forma` $\neq$ 'fm';
 - `self.fm_params` não seja `None`.
6. Aplicar filtro, caso `self.filtro_params` não seja `None`.
7. Aplicar o fator de intensidade `amp`.
8. Retornar o sinal gerado ou o par (t, y) , conforme o valor de `retorna_t`.

A implementação de efeitos por meio de `efeito_params` poderá ser tratada como evolução opcional do trabalho, de acordo com as escolhas da equipe.

Observações importantes

O método `gerar_nota` recebe a frequência da nota em Hertz. Portanto, caso a nota esteja representada por um número MIDI, ela deve ser convertida antes da chamada do método, por exemplo usando a função `midi2freq`.

O parâmetro `amp` deve ser aplicado ao sinal final da nota, de modo que diferentes intensidades extraídas do MIDI resultem em notas com amplitudes diferentes.

A aplicação de ADSR, AM, FM por *time warping* e filtro deve ser feita usando operações vetorizadas com NumPy sempre que possível.

Caso `self.forma` = 'fm', o dicionário `fm_params` deve conter os parâmetros necessários para a síntese FM. Caso contrário, a implementação deve indicar erro.

Exemplo conceitual de uso

O exemplo abaixo ilustra como um instrumento pode ser criado e usado para sintetizar uma nota. Ele não substitui a implementação do método `gerar_nota`, mas mostra o comportamento esperado da classe.

```
piano = Instrumento(
    nome='Piano-like',
    forma='dente',          # onda naturalmente rica em harmônicos
    n_harm=10,              # quantidade moderada de harmônicos
    adsr_params=(
        0.01,              # Ataque rápido
        0.2,               # Decaimento moderado
        0.4,               # Sustain baixo
        0.3                # Liberação moderada
    ),
    fase=0,
    unidade_fase='graus',
    am_params=None,
    fm_params=None,
    filtro_params={
        'cutoff_high': 3000 # passa-baixa leve
    },
    efeito_params=None
)

# Gera uma nota C4 por 1 segundo.
# A nota MIDI 60 corresponde aproximadamente a 261.63 Hz.
t, y = piano.gerar_nota(
    f=midi2freq(60),
    dur=1.0,
    retorna_t=True
)
```

ARQUIVO MICROPLAYER.PY

O arquivo `microplayer.py` contém estruturas relacionadas à leitura de arquivos MIDI, organização de eventos musicais, processamento de vozes e geração da waveform final.

As seguintes classes deverão ter suas implementações completadas pela equipe:

CLASSE ARQMIDI

```
class ArqMIDI:
```

A classe `ArqMIDI` é responsável por carregar um arquivo MIDI e convertê-lo em uma representação simplificada de eventos musicais, adequada ao processamento pelo MicroDAW.

A equipe **não precisa implementar um leitor MIDI completo do zero**. A biblioteca `music21` será usada para carregar e interpretar a estrutura básica do arquivo `.mid`. A tarefa da equipe é completar a conversão dos objetos musicais obtidos por `music21` para o formato de eventos usado pelo sintetizador.

Cada evento tonal extraído do MIDI deverá ser representado como uma tupla:

```
(frequencia, inicio, duracao, amplitude)
```

em que:

- `frequencia` é a frequência da nota, em Hertz;
 - `inicio` é o instante de início da nota, em segundos;
 - `duracao` é a duração da nota, em segundos;
 - `amplitude` é um valor em `[0, 1]`, derivado da *velocity* MIDI.
-

Construtor

```
def __init__(
    self,
    arq: str
) -> None:
```

O construtor da classe deve carregar o arquivo MIDI, identificar suas partes/vozes e armazenar os eventos musicais extraídos.

Parte da implementação será fornecida pronta, incluindo a leitura do arquivo com `music21`, a obtenção do andamento da música, o cálculo da duração total e a estrutura geral de armazenamento das partes.

A equipe deverá completar a conversão dos elementos musicais para eventos no formato usado pelo MicroDAW.

Parâmetros:

- `arq`: caminho para o arquivo MIDI a ser carregado.

Atributos principais criados pela classe:

- `self._rep`: representação interna do arquivo MIDI carregado pelo `music21`.
- `self.bpm`: andamento da música, em batidas por minuto.
- `self._spb`: duração de uma semínima, em segundos.
- `self.duracao`: duração total aproximada da música, em segundos.
- `self.parts`: lista de partes/vozes extraídas do arquivo MIDI.

Cada item de `self.parts` deverá ser um dicionário com a seguinte estrutura:

```
{
    'nome': nome_da_parte,
    'eventos': lista_de_eventos,
    'percussiva': False
}
```

Conversão dos eventos

Durante a leitura de cada parte/voz, a classe deve percorrer os elementos musicais retornados por `music21` e convertê-los para eventos simples.

A implementação deverá tratar pelo menos três tipos de elemento:

1. **Notas comuns** (`m21.note.Note`);
2. **Acordes** (`m21.chord.Chord`);
3. **Eventos percussivos ou sem altura definida** (`m21.note.Unpitched`).

Para notas comuns, deve ser criado um evento no formato:

```
(frequencia, inicio, duracao, amplitude)
```

Para acordes, cada nota do acorde deve gerar um evento independente, usando o mesmo instante de início, duração e amplitude.

Eventos percussivos não precisam ser sintetizados na versão básica do trabalho. Ao encontrar esse tipo de evento, a parte correspondente deve ser marcada como percussiva e o evento pode ser ignorado.

Comportamento esperado

A classe `ArqMIDI` deve:

- carregar um arquivo MIDI usando `music21`;
- obter o andamento da música em BPM;
- usar 120 BPM como valor padrão caso o arquivo não contenha marcação explícita de andamento;
- calcular a duração de uma semínima em segundos;
- calcular a duração total da música em segundos;
- identificar as partes/vozes presentes no arquivo MIDI;
- para cada parte, criar uma lista de eventos;
- converter offsets musicais para segundos;
- converter durações musicais para segundos;
- converter números MIDI de notas para frequências em Hertz;
- converter *velocity* MIDI para amplitude em `[0, 1]`;
- expandir acordes em múltiplos eventos;
- marcar partes percussivas quando necessário;
- ordenar os eventos de cada parte pelo instante de início.

Observações sobre tempo musical

No `music21`, os valores de `offset` e `duration.quarterLength` são expressos em semínimas.

Neste trabalho, os eventos do MicroDAW devem usar segundos. Portanto, é necessário converter esses valores usando a duração de uma semínima em segundos:

```
segundos_por_beat = 60 / bpm
```

Assim:

- o início de uma nota em segundos pode ser obtido a partir de seu `offset`;
- a duração de uma nota em segundos pode ser obtida a partir de `duration.quarterLength`.

Observações sobre intensidade

A *velocity* MIDI normalmente varia de 0 a 127.

Para uso no sintetizador, essa informação deve ser convertida para uma amplitude em `[0, 1]`.

Caso a *velocity* não esteja disponível, deve ser usado um valor padrão de 64.

Método `getPartList`

```
def getPartList(  
    self  
) -> list[tuple]:
```

Retorna uma lista com informações resumidas sobre as partes/vozes extraídas do arquivo MIDI.

Cada item da lista retornada deve ter o formato:

```
(nome, percussiva, quantidade_de_eventos)
```

em que:

- nome é o nome da parte ou voz;
- percussiva indica se a parte contém eventos percussivos;
- quantidade_de_eventos indica quantos eventos tonais foram extraídos.

Esse método será fornecido pronto.

Método `getPart`

```
def getPart(  
    self,  
    chave: int  
) -> list[tuple]:
```

Retorna a lista de eventos de uma parte/voz específica.

Parâmetros:

- chave: índice da parte/voz desejada.

Retorno:

- Lista de eventos da parte selecionada.

Cada evento retornado deve estar no formato:

```
(frequencia, inicio, duracao, amplitude)
```

Esse método será fornecido pronto.

Exemplo conceitual de uso

```
midi = ArqMIDI('midi/exemplo.mid')  
  
print(midi.bpm)  
print(midi.duracao)  
print(midi.getPartList())  
  
eventos_voz_0 = midi.getPart(0)
```

Nesse exemplo, `eventos_voz_0` será uma lista de eventos musicais extraídos da primeira parte/voz do arquivo MIDI.

CLASSE `PLAYER`

```
class Player:
```

A classe `Player` representa o motor de renderização offline do MicroDAW.

Ela é responsável por integrar os elementos principais do sistema:

- o arquivo MIDI carregado pela classe `ArqMIDI`;
- os instrumentos sintéticos definidos pela classe `Instrumento`;
- os eventos musicais extraídos de cada parte/voz;
- os buffers de áudio intermediários;
- a waveform final, pronta para reprodução ou exportação.

Enquanto a classe `ArqMIDI` transforma o arquivo `.mid` em uma lista de eventos, a classe `Player` transforma esses eventos em áudio sintetizado.

Inicialização

```
def __init__(
    self
) -> None:
```

Inicializa o objeto Player.

Ao ser criado, o player não possui ainda um arquivo MIDI associado, instrumentos definidos ou buffers de áudio preenchidos.

A inicialização deve preparar a estrutura interna da classe para receber posteriormente um arquivo MIDI e seus instrumentos associados.

Método `_limpaEstr`

```
def _limpaEstr(
    self
) -> None:
```

Reinicializa as estruturas internas do player.

Esse método é usado para limpar instrumentos, buffers, ganhos e taxa de amostragem sempre que um novo arquivo for carregado ou quando for necessário reiniciar o estado do objeto.

Comportamento esperado:

O método deve redefinir, pelo menos:

- a lista de instrumentos;
- os buffers de áudio;
- a taxa de amostragem;
- os ganhos associados às vozes.

Esse método será fornecido pronto.

Método `setArq`

```
def setArq(
    self,
    nome_arq: str,
    sr: int = 44100
) -> None:
```

Define o arquivo MIDI a ser processado pelo player.

Esse método deve criar internamente um objeto `ArqMIDI`, responsável por carregar o arquivo e extrair suas partes/vozes e eventos musicais.

Parâmetros:

- `nome_arq`: caminho para o arquivo MIDI.
- `sr`: taxa de amostragem a ser usada no processamento.

Comportamento esperado:

O método deve:

- limpar estruturas anteriores do player;
- carregar o arquivo MIDI usando a classe `ArqMIDI`;
- definir a taxa de amostragem a ser usada no processamento.

Esse método será fornecido pronto.

Método `setSr`

```
def setSr(  
    self,  
    sr: int  
) -> None:
```

Define a taxa de amostragem usada pelo player.

Parâmetros:

- sr: taxa de amostragem, em amostras por segundo.

Comportamento esperado:

O método deve validar se sr é um inteiro positivo e armazenar o valor na estrutura interna do objeto.

Esse método será fornecido pronto.

Método `setInstrumentos`

```
def setInstrumentos(  
    self,  
    instrumentos: list  
) -> None:
```

Associa instrumentos sintéticos às partes/vozes do arquivo MIDI carregado.

Cada parte/voz do arquivo MIDI deverá ter um instrumento correspondente. Assim, o número de instrumentos informados deve ser igual ao número de partes/vozes extraídas pela classe `ArqMIDI`.

Cada item da lista `instrumentos` poderá ser:

- um objeto da classe `Instrumento`;
- uma tupla no formato `(instrumento, ganho)`, em que:
 - instrumento é um objeto da classe `Instrumento`;
 - ganho é um fator multiplicativo usado na mixagem daquela voz.

Parâmetros:

- instrumentos: lista de instrumentos ou pares `(instrumento, ganho)`.

Comportamento esperado:

O método deve:

- verificar se há um arquivo MIDI carregado;
- verificar se o número de instrumentos corresponde ao número de partes/vozes;
- validar se cada instrumento informado é uma instância de `Instrumento`;
- armazenar os instrumentos;
- armazenar os ganhos de cada voz;
- criar os buffers de áudio intermediários.

Os buffers devem ser armazenados em um array NumPy bidimensional, com:

- uma linha para cada parte/voz;
- colunas suficientes para armazenar a música inteira.

Recomenda-se adicionar um pequeno tempo extra ao final do buffer, para acomodar releases, filtros ou pequenas sobras de sinal.

Método `processa`

```
def processa(  
    self,  
    canais: int = 2,  
    verbose: bool = False  
) -> np.ndarray:
```

Processa o arquivo MIDI e gera o áudio sintetizado.

Esse método é uma das partes centrais do MicroDAW. Ele deve percorrer os eventos musicais de cada voz, gerar as notas correspondentes usando os instrumentos associados e posicionar cada nota corretamente no buffer da voz.

Parâmetros:

- **canais**: define o tipo de saída desejado:
 - 0: retorna os buffers separados por voz;
 - 1: retorna áudio mono;
 - 2: retorna áudio estéreo.
- **verbose**: se True, permite imprimir informações sobre o processamento.

Retorno:

- Um array NumPy contendo o áudio sintetizado, de acordo com o número de canais solicitado.

Comportamento esperado:

O método deve:

- verificar se há um arquivo MIDI carregado;
- verificar se os instrumentos foram definidos;
- verificar se os buffers foram criados;
- percorrer todas as partes/vozes do arquivo MIDI;
- obter os eventos de cada parte;
- obter o instrumento correspondente àquela parte;
- para cada evento, sintetizar a nota usando `Instrumento.gerar_nota`;
- converter o instante de início da nota, em segundos, para índice de amostra;
- calcular o trecho do buffer em que a nota será inserida;
- somar o sinal da nota ao buffer da voz correspondente;
- tratar corretamente notas que ultrapassem o tamanho final do buffer;
- normalizar ou controlar a amplitude de cada voz, caso essa estratégia seja adotada;
- retornar a saída produzida por `getWav`.

Ao sintetizar uma nota, a taxa de amostragem usada pelo player deve ser repassada ao instrumento. Assim, a chamada ao método `gerar_nota` deve usar a taxa de amostragem armazenada no próprio `Player`.

Método `getWav`

```
def getWav(
    self,
    canais: int = 2
) -> np.ndarray:
```

Gera a waveform final a partir dos buffers processados.

Esse método deve combinar as vozes sintetizadas, aplicar ganhos e retornar a saída no formato solicitado.

Parâmetros:

- **canais**: define o tipo de saída desejado:
 - 0: retorna os buffers separados por voz;
 - 1: retorna áudio mono;
 - 2: retorna áudio estéreo.

Retorno:

- Um array NumPy contendo:
 - os buffers separados por voz, se `canais == 0`;
 - uma waveform mono, se `canais == 1`;
 - uma waveform estéreo, se `canais == 2`.

Comportamento esperado:

O método deve:

- validar se há arquivo MIDI carregado;
- validar se os instrumentos foram definidos;
- validar se os buffers existem;
- aplicar os ganhos de cada voz;
- evitar modificar destrutivamente os buffers originais ao aplicar ganhos;
- se canais = 0, retornar os buffers separados por voz;
- se canais = 1 ou canais = 2, somar as vozes em uma waveform única;
- normalizar a mistura final para evitar clipping;
- se canais = 1, retornar a waveform mono;
- se canais = 2, retornar uma versão estéreo simples.

Para a saída estéreo simples, é aceitável duplicar a mesma waveform mono nos dois canais.

Observações sobre buffers

Os buffers de áudio devem representar o sinal de cada voz ao longo do tempo.

Ao inserir uma nota no buffer, é necessário converter seu instante de início para índice de amostra:

```
indice_inicial = int(inicio_em_segundos * sr)
```

A partir desse índice, o sinal da nota deve ser somado ao trecho correspondente do buffer.

Caso a nota ultrapasse o tamanho final do buffer, a implementação deve inserir apenas a parte que couber.

Observações sobre mixagem

A mixagem final consiste em combinar as vozes processadas em uma única waveform.

Uma estratégia simples consiste em:

1. aplicar o ganho de cada voz;
2. somar os buffers das vozes;
3. normalizar o resultado final.

A normalização final deve evitar que a soma das vozes gere amplitudes excessivas e cause distorção por clipping.

Exemplo conceitual de uso

```
pl = Player()

pl.setArq('midi/exemplo.mid', sr=44100)

pl.setInstrumentos([
    instrumento_voz_0,
    instrumento_voz_1,
    instrumento_voz_2
])

audio = pl.processa(canais=2, verbose=True)
```

Nesse exemplo, o arquivo MIDI é carregado, cada voz recebe um instrumento sintético e o player gera uma waveform estéreo com a música sintetizada.

9.3. Partes a implementar do zero

Algumas funções deverão ser implementadas integralmente pela equipe. Nesses casos, serão fornecidas apenas as assinaturas esperadas, a descrição do comportamento desejado, os parâmetros e os retornos.

As funções desta seção devem ser implementadas de forma compatível com o restante do sistema, pois serão chamadas por outras funções, classes ou métodos dos arquivos-base.

ARQUIVO MICROSYNTH.PY

O arquivo `microsynth.py` concentra as funções ligadas diretamente à síntese e à transformação de sinais sonoros.

As seguintes funções deverão ser implementadas do zero pela equipe:

`freq2midi`

```
def freq2midi(f: float) -> int:
```

Converte uma frequência, em Hertz, para o número MIDI correspondente.

Essa função realiza a operação inversa de `mid2freq`. Ela deverá considerar o padrão em que a nota MIDI 69 corresponde à frequência de 440 Hz, isto é, a nota Lá 4 (A4).

A conversão deverá usar a relação logarítmica entre frequência e altura musical no sistema temperado de 12 semitons por oitava.

Parâmetros:

- `f`: frequência da nota, em Hertz. Deve ser um valor numérico positivo.

Retorno:

- Um número inteiro correspondente à nota MIDI mais próxima da frequência informada.

Comportamento esperado:

- A função deve retornar um inteiro.
- Frequências inválidas, como valores nulos ou negativos, devem ser tratadas adequadamente.
- A implementação deve usar operações matemáticas com NumPy ou funções equivalentes.

Exemplos conceituais:

- `freq2midi(440)` deve retornar 69.
- `freq2midi(261.63)` deve retornar um valor próximo ao MIDI correspondente à nota Dó 4 (C4).

`fm`

```
def fm(
    dur: float,
    f_c: float,
    f_m: float,
    I: float,
    tipo_fm: str = 'const',
    fase: float = 0.0,
    unidade_fase: str = 'graus',
    sr: int = 44100,
    retorna_t: bool = False
) -> np.ndarray | tuple[np.ndarray, np.ndarray]:
```

Gera um sinal por modulação de frequência, ou FM (*Frequency Modulation*).

Na síntese FM, uma frequência portadora é modulada por uma segunda frequência, chamada frequência moduladora. Dependendo dos parâmetros escolhidos, o resultado pode variar de um vibrato sutil até um timbre complexo, com surgimento de novas componentes espectrais.

A forma básica esperada é:

$$y(t) = \sin\pi(2 \cdot f_c \cdot t + I \cdot \sin\pi(2 \cdot f_m \cdot t) + fase)$$

em que:

- `f_c` é a frequência da portadora;
- `f_m` é a frequência da moduladora;
- `I` é o índice de modulação.

Parâmetros:

- **dur**: duração do sinal, em segundos.
- **f_c**: frequência da onda portadora, em Hertz.
- **f_m**: frequência da onda moduladora.
- **I**: índice de modulação, responsável pela intensidade da modulação.
- **tipo_fm**: modo de interpretação da frequência moduladora.
 - 'const': usa **f_m** diretamente como frequência de modulação, em Hertz.
 - 'mult': interpreta **f_m** como multiplicador da frequência portadora.
- **fase**: fase inicial do sinal.
- **unidade_fase**: unidade usada no parâmetro **fase**. Deve aceitar 'graus' ou 'rad'.
- **sr**: taxa de amostragem, em amostras por segundo.
- **retorna_t**: indica se a função deve retornar também o vetor de tempo.

Retorno:

- Se **retorna_t** for False, retorna apenas o sinal sintetizado **y**, como array NumPy.
- Se **retorna_t** for True, retorna uma tupla (**t**, **y**), em que:
 - **t** é o vetor de tempo;
 - **y** é o sinal sintetizado por FM.

Comportamento esperado:

- A função deve gerar internamente o vetor de tempo usando a duração e a taxa de amostragem.
- A fase deve ser convertida corretamente caso seja informada em graus.
- O parâmetro **tipo_fm** deve alterar a interpretação de **f_m** conforme descrito.
- A implementação deve usar operações vetorizadas com NumPy.
- A função deve retornar arrays compatíveis com o restante do sistema.

am

```
def am(
    y: np.ndarray,
    f_mod: float,
    I: float = 0.5,
    sr: int = 44100
) -> np.ndarray:
```

Aplica modulação de amplitude, ou AM (*Amplitude Modulation*), a um sinal já existente.

Na modulação de amplitude, a amplitude do sinal original varia ao longo do tempo de acordo com um sinal modulador. Em síntese musical, esse processo pode produzir desde um tremolo suave até alterações perceptíveis no timbre, dependendo da frequência e da intensidade da modulação.

Uma forma simples de modulação de amplitude é:

$$y_{\text{mod}}(t) = y(t) \cdot (1 + I \cdot \sin(\pi(2 \cdot f_{\text{mod}} \cdot t)))$$

Parâmetros:

- **y**: sinal de entrada, representado como um array NumPy.
- **f_mod**: frequência da modulação, em Hertz.
- **I**: intensidade da modulação.
- **sr**: taxa de amostragem, em amostras por segundo.

Retorno:

- Um array NumPy contendo o sinal resultante após a aplicação da modulação de amplitude.

Comportamento esperado:

- A função deve gerar um vetor de tempo compatível com o tamanho do sinal de entrada.
- A modulação deve ser aplicada de forma vetorizada.
- O sinal de saída deve ter o mesmo tamanho do sinal de entrada.
- A função não deve alterar o array original de forma destrutiva, a menos que essa decisão seja claramente justificada.

- Valores altos de I podem amplificar significativamente o sinal; a equipe deve considerar isso ao usar a função em instrumentos ou no processo de mixagem.

ARQUIVO `MICROPLAYER.PY`

Não há funções ou classes a serem implementadas integralmente do zero neste arquivo.

O arquivo `microplayer.py` conterá partes fornecidas prontas e partes parcialmente implementadas, que deverão ser completadas pela equipe conforme indicado na seção de implementação parcial.

ARQUIVO `MICROVIEW.PY`

Não há funções a serem implementadas do zero neste arquivo.

O arquivo `microview.py` será fornecido integralmente pronto e poderá ser usado livremente para visualização e análise dos sinais gerados.

9.4. Evoluções naturais

- No seu trabalho, implemente **pelo menos duas evoluções** listadas a seguir, à escolha do grupo
- Não serão cobradas todas implementações, mas caso vocês desejem um resultado mais realista, são recomendadas

A. SEPARAR *SUSTAIN* DE *RELEASE* (ADSR “DE VERDADE”)

No modelo simplificado usado inicialmente, a duração da nota coincide com a duração total do envelope. Em um ADSR mais próximo do funcionamento real, a duração da nota corresponde ao tempo até o início do *release*.

Nesse modelo:

- o *attack* e o *decay* acontecem após o início da nota;
- o *sustain* permanece até o fim da duração musical da nota;
- o *release* acontece depois disso;
- portanto, o sinal gerado por uma nota pode durar mais do que a duração indicada pelo evento MIDI.

A equipe deverá adaptar a geração do envelope e o posicionamento das notas no buffer para lidar corretamente com esse comportamento.

Dica: nesse modelo, o tamanho final do sinal de uma nota pode ser maior que $\text{dur} * \text{sr}$.

💡 Esse modelo é mais fiel ao funcionamento de sintetizadores reais, mas exige maior cuidado com o gerenciamento do *buffer* (a nota pode ultrapassar sua duração original).

B. ADSR EXPONENCIAL

O ADSR inicial usa transições lineares, geralmente construídas com `linspace`. Essa abordagem é simples, mas pode soar artificial em alguns casos.

Nesta evolução, a equipe deverá substituir uma ou mais fases lineares do envelope por curvas não lineares, especialmente curvas de comportamento exponencial.

A implementação deve permitir controlar a curvatura das fases modificadas por meio de um ou mais parâmetros. A equipe deverá explicar como esses parâmetros afetam o resultado sonoro.

Possíveis pontos de intervenção:

- *attack* com crescimento rápido no início e desaceleração próxima do pico;
- *decay* com queda mais rápida no início e mais suave no final;
- *release* com decaimento mais natural.

Dica: expressões baseadas em $\exp(-k \cdot t)$, com t normalizado entre 0 e 1, podem ser usadas para construir curvas desse tipo.

C. ENVELOPE DEPENDENTE DE VELOCITY / AMP

Na versão básica, a intensidade da nota é usada apenas para multiplicar a amplitude final do sinal. Nesta evolução, a intensidade também deverá influenciar o comportamento temporal da nota.

A equipe deverá modificar pelo menos dois parâmetros do envelope em função da intensidade da nota. Por exemplo, notas mais fortes podem ter:

- ataque mais rápido;
- decaimento mais curto;
- maior nível de sustentação;
- liberação mais intensa ou mais curta;
- maior brilho, caso a equipe combine esta evolução com filtros ou harmônicos.

A equipe deverá justificar as escolhas feitas e explicar o efeito perceptivo esperado.

Atenção: a implementação deve garantir que os parâmetros modificados continuem em faixas válidas. Evite tempos negativos, níveis de sustentação fora do intervalo $[0, 1]$ ou envelopes incompatíveis com a duração da nota.

💡 Essa abordagem adiciona expressividade ao som gerado, aproximando o resultado de uma execução musical mais natural.

D. CONTROLE DE VOZES SIMULTÂNEAS (POLYPHONY LIMIT)

Sintetizadores reais costumam ter um limite máximo de notas simultâneas. Nesta evolução, a equipe deverá implementar um limite de polifonia para evitar que um número excessivo de notas soe ao mesmo tempo.

Quando o limite for atingido, o sistema deverá escolher uma nota ativa para ser removida, atenuada ou finalizada antecipadamente.

A estratégia de *note stealing* deve ser definida pela equipe. Algumas possibilidades:

- remover a nota de menor intensidade;
- remover a nota mais próxima do fim;
- remover a nota mais grave;
- remover a nota mais aguda;
- aplicar um *fade-out* curto na nota escolhida.

A equipe deverá explicar a estratégia escolhida e comentar seus efeitos sobre o resultado sonoro.

Atenção: não use simplesmente a estratégia de remover a nota mais antiga. A proposta desta evolução é pensar em uma política própria de gerenciamento de vozes.

E. SINTETIZADOR PERCUSSIVO

Eventos percussivos não são modelados da mesma forma que notas tonais. Em muitos casos, sons percussivos são caracterizados por transientes curtos, ruído, envelopes rápidos e, às vezes, componentes tonais de curta duração.

Nesta evolução, a equipe deverá implementar pelo menos um tipo de som percussivo sintético e integrá-lo ao processamento da música.

Possíveis estratégias:

- ruído branco com envelope curto;
- ruído filtrado para sons mais agudos;
- senoide grave com frequência decrescente para simular bumbo;
- mistura de ruído e componente tonal para simular caixa;
- diferentes envelopes para diferentes tipos de percussão.

A equipe deverá explicar como o som percussivo foi modelado e como ele foi integrado ao processamento das vozes ou eventos MIDI.

10. Observações e lembretes finais

- Este trabalho combina programação, processamento de sinais e criatividade musical. O resultado final não precisa soar como um instrumento real, mas deve demonstrar uso consciente dos conceitos estudados.
- Mais importante do que produzir um áudio “bonito” é construir uma solução tecnicamente correta, organizada e capaz de evidenciar como diferentes escolhas de síntese afetam o som produzido.
- Ao calcular a frequência das notas MIDI (há isso pronto na `music21`. Você pode usar para **conferir seus resultados**, mas **não** como implementação.
- Reforçando: o programa deve gerar uma única waveform final correspondente à soma de todas as vozes, pronta para reprodução. Para simplificar, use o formato `.wav`. A partir dele, você pode:
 - Salvar o arquivo e executar em um player de áudio à parte
 - Tentar acoplar alguma implementação de player em Python, usando bibliotecas específicas, como `sounddevice`
 - Se sua solução de interface for em um Notebook, pode usar o player nativo do IPython, como ilustrado diversas vezes em sala
- As notas devem ser posicionadas corretamente no tempo, respeitando seus offsets e durações.
- Lembre-se de normalizar áudio para não “estourar”

11. Entrega

1. Os entregáveis finais serão especificados posteriormente. Em princípio, a entrega deverá incluir:
 - código-fonte da solução;
 - arquivo(s) MIDI extra(s), se incluir algum por conta própria (apenas os extras, se houver);
 - relatório em PDF, contendo:
 - **IDENTIFICAÇÃO** dos membros da equipe (nome e matrícula)
 - instruções de execução, para que o professor possa gerar e executar os arquivos de áudio (`.wav`) produzidos;
 - breve descrição dos instrumentos criados;
 - breve explicação das escolhas técnicas adotadas.
2. Compacte todos os arquivos **em um único pacote** `.zip`
 - Caso o arquivo `.zip` ultrapasse o limite máximo permitido pelo Moodle (20 MB), proceda assim:
 1. Suba o pacote para uma pasta do Google Drive de um dos membros da equipe
 2. Mantenha compactado em um único pacote, para facilitar o download por parte do professor
 3. Compartilhe a pasta com o professor (`marcosh.ribeiro@ufv.br`)
 4. Crie um arquivo `.txt` contendo o link para a pasta compartilhada
 5. Envie o arquivo `.txt` como resposta à atividade
3. Envie pelo Moodle até o dia **13/06/206, às 23:59:59**
4. O Moodle só aceitará arquivos nos formatos `.zip` ou `.txt`
5. Apenas um membro da equipe precisa fazer o envio

12. Inspirações para criação de instrumentos sintéticos

Aqui estão fornecidos alguns modelos básicos de instrumentos apenas como referência e para testes iniciais. Esses modelos não contam como instrumentos próprios da equipe. Para atender ao requisito de criação de instrumentos próprios, a equipe deverá propor combinações próprias de parâmetros e técnicas de síntese. Alterar apenas valores numéricos de um modelo fornecido não será considerado suficiente. Cada instrumento próprio deve ser acompanhado de uma breve justificativa técnica, explicando quais características sonoras eram desejadas e quais recursos de síntese foram usados para obtê-las.

- As descrições a seguir apresentam ideias conceituais para a criação de instrumentos sintéticos. Elas não devem ser entendidas como receitas prontas, mas como pontos de partida para pesquisa, experimentação e projeto de timbres.
- A equipe poderá se inspirar nesses exemplos, pesquisar outras estratégias e combinar técnicas diferentes. O importante é que os instrumentos próprios sejam compreendidos, justificados e integrados corretamente ao MicroDAW.

12.1 Sino

Características típicas:

- ataque muito rápido;
- decaimento longo;
- presença de parciais inarmônicas;
- brilho metálico;
- sensação de reverberação ou prolongamento.

Possíveis estratégias de síntese:

Aspecto	Estratégia possível
Parciais inarmônicas	usar frequências que não sejam múltiplos inteiros exatos da fundamental
Ataque rápido	envelope com <i>attack</i> muito curto
Decaimento longo	<i>release</i> ou decaimento exponencial mais prolongado
Brilho metálico	componentes agudas com menor amplitude
Complexidade tímbrica	FM ou soma de componentes senoidais desalinhadas

Um sino pode ser modelado por síntese aditiva com parciais inarmônicas, por FM ou por uma combinação das duas abordagens.

12.2 Violão ou corda dedilhada

Características típicas:

- ataque curto, associado ao dedo, unha ou palheta;
- decaimento relativamente rápido;
- harmônicos com amplitudes decrescentes;
- harmônicos agudos tendem a desaparecer mais rapidamente;
- pode haver pequeno ruído no ataque.

Possíveis estratégias de síntese:

Aspecto	Estratégia possível
Corda dedilhada	envelope com ataque rápido e decaimento natural
Corpo harmônico	soma de harmônicos com amplitudes decrescentes
Perda de brilho	harmônicos agudos com decaimento mais rápido
Ruído inicial	ruído curto no início da nota
Maior realismo	pesquisar algoritmo de Karplus-Strong

A equipe pode pesquisar modelos de cordas dedilhadas e propor uma implementação própria compatível com a classe `Instrumento`.

12.3 Piano

Características típicas:

- ataque percussivo muito definido;
- espectro rico em harmônicos;
- decaimento em mais de uma fase;
- leve inarmonicidade;
- variação perceptível com a intensidade da nota;
- em pianos reais, algumas notas possuem múltiplas cordas levemente desafinadas entre si.

Possíveis estratégias de síntese:

Aspecto	Estratégia possível
Ataque percussivo	envelope com <i>attack</i> muito curto
Brilho	maior presença de harmônicos
Decay complexo	combinação de decaimentos rápidos e lentos
Inarmonicidade	pequenos desvios nas frequências dos harmônicos
Variação com intensidade	notas mais fortes com mais brilho ou ataque mais marcado
Múltiplas cordas	componentes próximas com pequenas desafinações

O objetivo não é reproduzir um piano real com perfeição, mas construir um timbre que explore algumas dessas características.

12.4 Percussão

Características típicas:

- predominância de transientes;
- envelopes curtos;
- presença de ruído;
- frequência instável ou pouco definida;
- em alguns casos, mistura de componente tonal e ruído.

Possíveis estratégias:

Som	Estratégia possível
Bumbo	componente grave com frequência decrescente e envelope curto
Caixa	mistura de ruído e componente tonal curta
Hi-hat	ruído filtrado em altas frequências com decaimento muito rápido
Prato	ruído agudo com decaimento mais longo
Tom	componente tonal curta com decaimento médio

A equipe pode implementar um ou mais sons percussivos, desde que explique como cada som foi construído.

12.5. Observação final sobre os modelos

Os modelos descritos nesta seção são apenas inspirações. Eles indicam características acústicas e possíveis caminhos de síntese, mas não definem uma implementação obrigatória.

A criação dos instrumentos próprios será avaliada considerando:

- uso consciente dos conceitos de síntese;
- coerência entre objetivo sonoro e parâmetros escolhidos;
- integração correta com o MicroDAW;
- clareza da explicação apresentada pela equipe;
- diferença efetiva em relação aos modelos básicos fornecidos.